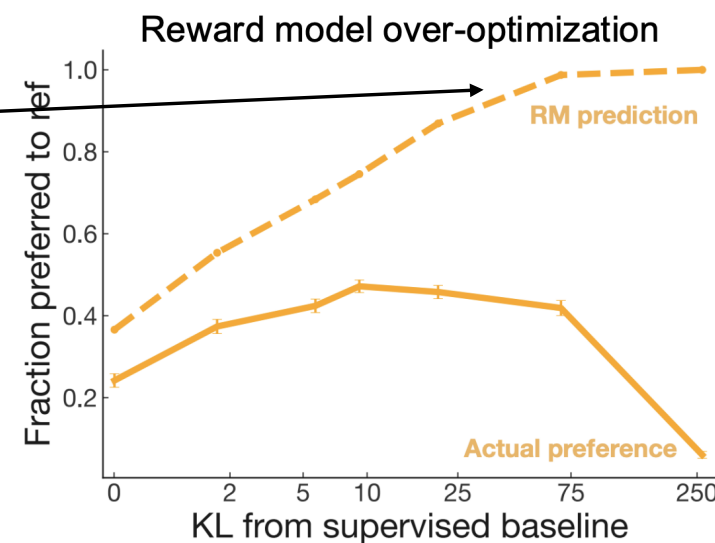
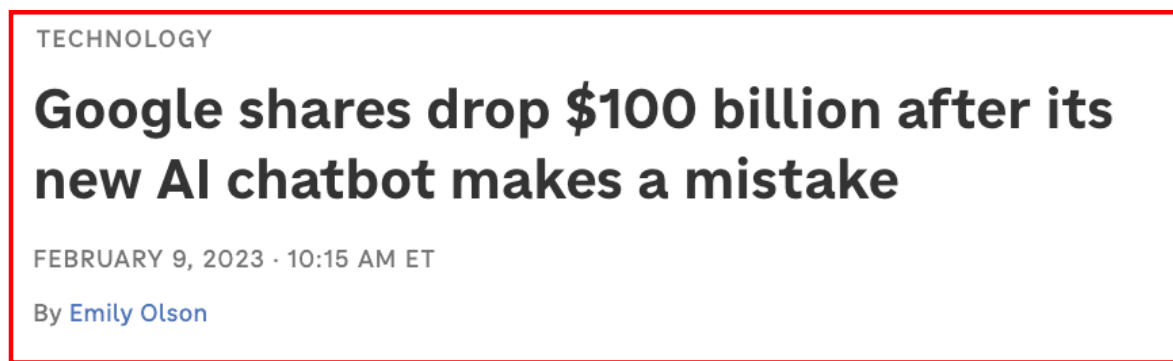


Limitations of RL + Reward Model

- “Reward hacking” is a common problem in RL



- Human preferences can be noisy, and a “reward model” trained with this data can be unreliable. The KL divergence penalty can mitigate such negative effect:

$$R(s) = RM_{\phi}(s) - \beta \log \left(\frac{p_{\theta}^{RL}(s)}{p^{PT}(s)} \right)$$

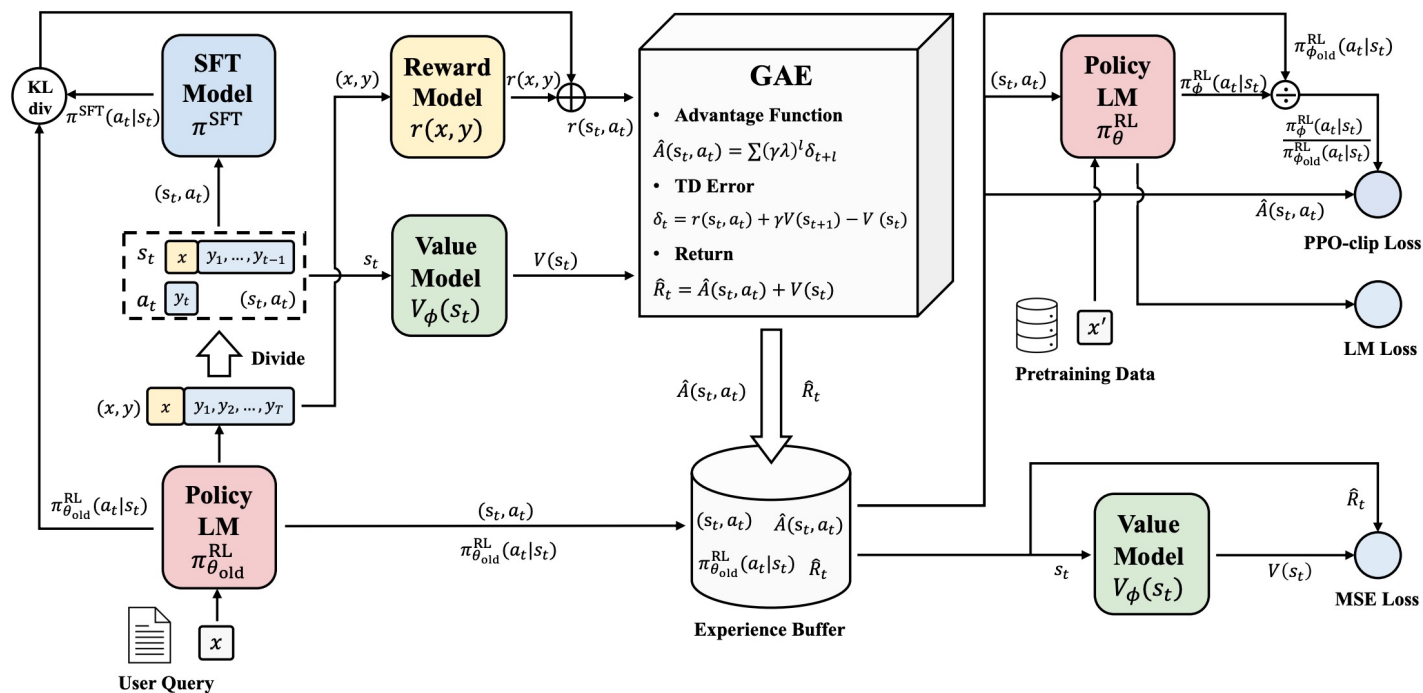
Stay close to the reference pre-trained model, according to distribution divergence.

Image source: <https://www.npr.org/2023/02/09/1155650909/google-chatbot--error-bard-shares>

[1] Stiennon, Nisan, et al. "Learning to summarize with human feedback." Advances in neural information processing systems 33 (2020): 3008-3021.

RLHF with PPO can be quite complex

RLHF using PPO is quite **expensive and complex to implement**.



- PPO is an **advantage** actor-critic (AC2) algorithm:
 - **Actor**: Policy LM
 - **Critic**: Value Model of the same architecture of the actor but with delayed parameters, using for calculating advantage.

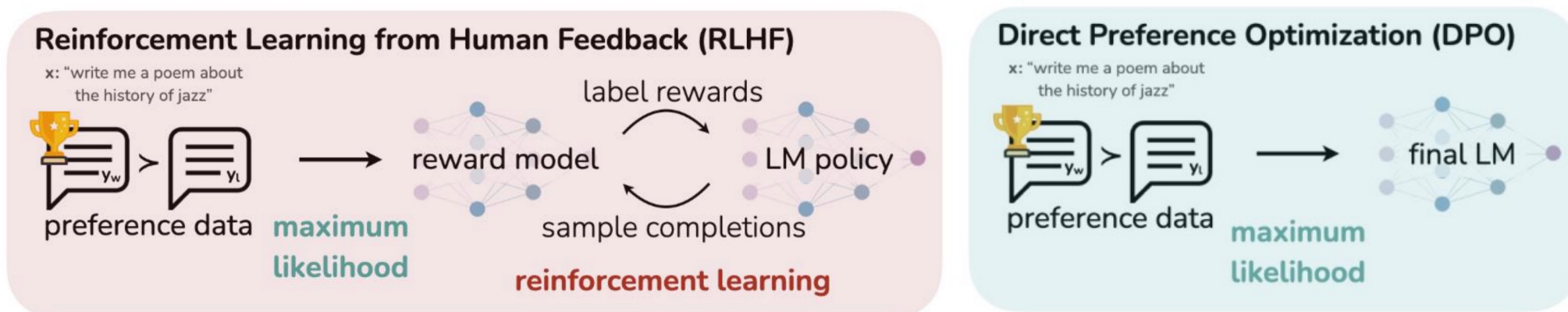
[2] Zheng, Rui, et al. "Secrets of rlhf in large language models part i: Ppo." arXiv preprint arXiv:2307.04964 (2023).

Why using advantages

- Consider several rollouts with delayed rewards: 10, 100, 1000
 - Does 10 means positive and desirable?
- Compare to the average $(10+100+1000)/3=370$ to compute the advantages of the rollouts:
 - $10-370=-360$ (really bad rollout)
 - $100-370=-270$ (bad rollout)
 - $1000-370=630$ (great rollout, but not as extreme as 1000)
- Using the advantages can identify the true value of a rollout.
 - Similarly, it works for individual (s,a) pairs.

DPO Motivation

So, is there any way to do away with the cumbersome reinforcement learning reward model and optimize the model directly using preference data?



Direct Preference Optimization (DPO): while PPO optimizes this objective using a reinforcement learning approach, DPO derives a closed-form solution to the reward function for simpler MLE optimization. Their goals are aligned even with different forms.

[3] Rafailov, Rafael, et al. "Direct preference optimization: Your language model is secretly a reward model." Advances in neural information processing systems 36 (2023): 53728-53741.

Solve optimization objective of RLHF

RLHF aims to train a policy model p_{θ}^{RL} such that the responses $\hat{y}$ receive the highest possible scores from the reward model RM_{ϕ}

$$\mathbb{E}_{\hat{y} \sim p_{\theta}^{RL}(\hat{y}|x)} [RM_{\phi}(x, \hat{y}) - \beta \log \left(\frac{p_{\theta}^{RL}(\hat{y}|x)}{p^{PT}(\hat{y}|x)} \right)]$$

Denote $p_{\theta}^{RL}(\hat{y}|x)$ as $\pi(y)$. The original objective function can be written as:

$$J(\pi) = \sum_y \pi(y) \left[RM(x, y) - \beta \log \frac{\pi(y)}{p^{PT}(y)} \right]$$

$\pi(y)$ is a probability distribution

$$\sum_y \pi(y) = 1 \implies \sum_y \pi(y) - 1 = 0$$

To solve this constrained extremum problem, introduce a Lagrange multiplier λ to incorporate the constraint in the objective function and obtain the Lagrangian function:

$$\mathcal{L}(\pi, \lambda) = \sum_y \pi(y) \left[RM(x, y) - \beta \log \frac{\pi(y)}{p^{PT}(y)} \right] + \lambda \left(1 - \sum_y \pi(y) \right)$$

Solve the optimization objective of RLHF

To find the extreme point, take the partial derivative of $\mathcal{L}$ with respect to the variable $\pi(y)$

$$\begin{aligned}\frac{\partial \mathcal{L}}{\partial \pi(y)} &= RM(x, y) - \beta \left(\log \frac{\pi(y)}{p^{PT}(y)} + \pi(y) \cdot \frac{p^{PT}(y)}{\pi(y)} \cdot \frac{1}{p^{PT}(y)} \right) - \lambda \\ &= RM(x, y) - \beta \left(\log \frac{\pi(y)}{p^{PT}(y)} + 1 \right) - \lambda\end{aligned}$$

Set the partial derivative to 0. The optimal policy p^* should satisfy

$$RM(x, y) - \beta \log \frac{p^*(y)}{p^{PT}(y)} - \beta - \lambda = 0$$

We keep the terms containing $p^*(y)$ on the left and move the rest to the right:

$$\beta \log \frac{p^*(y)}{p^{PT}(y)} = RM(x, y) - \beta - \lambda$$

Solve the optimization objective of RLHF

Divide both sides by β :

$$\log \frac{p^*(y)}{p^{PT}(y)} = \frac{1}{\beta} RM(x, y) - \frac{\beta + \lambda}{\beta}$$

Take the exponent from both sides, and remove $\log$:

$$\frac{p^*(y)}{p^{PT}(y)} = \exp \left(\frac{1}{\beta} RM(x, y) \right) \exp \left(-\frac{\beta + \lambda}{\beta} \right)$$

Multiplying $p^{PT}(y)$ gives us the appearance of the optimal distribution:

$$p^*(y) = p^{PT}(y) \exp \left(\frac{1}{\beta} RM(x, y) \right) \exp \left(-\frac{\beta + \lambda}{\beta} \right)$$

We sum both sides of the equation over all possible y , to obtain a constant 1 of total probability:

$$\sum_y p^*(y) = \sum_y \left[p^{PT}(y) \exp \left(\frac{1}{\beta} RM(x, y) \right) \exp \left(-\frac{\beta + \lambda}{\beta} \right) \right]$$

Solve the optimization objective of RLHF

On the right side, since $\exp\left(-\frac{\beta+\lambda}{\beta}\right)$ is a constant independent of y , move it outside the summation:

$$1 = \exp\left(-\frac{\beta+\lambda}{\beta}\right) \sum_y p^{PT}(y) \exp\left(\frac{1}{\beta}RM(x, y)\right) \quad \exp\left(\frac{\beta+\lambda}{\beta}\right) = \sum_y p^{PT}(y) \exp\left(\frac{1}{\beta}RM(x, y)\right)$$

Substituting back into $p^*(y)$ obtained in previous steps, we get:

$$p^*(y) = p^{PT}(y) \exp\left(\frac{1}{\beta}RM(x, y)\right) \exp\left(-\frac{\beta+\lambda}{\beta}\right) = \frac{1}{Z(x)} p^{PT}(\hat{y}|x) \exp\left(\frac{1}{\beta}RM(x, \hat{y})\right)$$

At the variational extremum of the objective of RLHF, we obtain a theoretical closed-form solution.

$$\mathbb{E}_{\hat{y} \sim p_{\theta}^{RL}(\hat{y}|x)} [RM_{\phi}(x, \hat{y}) - \beta \log \left(\frac{p_{\theta}^{RL}(\hat{y}|x)}{p^{PT}(\hat{y}|x)} \right)] \xrightarrow{\text{at its theoretical optima}} p^*(\hat{y}|x) = \frac{1}{Z(x)} p^{PT}(\hat{y}|x) \exp\left(\frac{1}{\beta}RM(x, \hat{y})\right)$$

Solve the optimization objective of RLHF

Note that the right-hand side depends on the known reference model and reward score, and the difficult-to-compute *partition function* $Z(x)$

$$p^*(\hat{y}|x) = \frac{1}{Z(x)} p^{PT}(\hat{y}|x) \exp\left(\frac{1}{\beta} RM(x, \hat{y})\right) \quad \exp\left(-\frac{\beta + \lambda}{\beta}\right) = \frac{1}{Z(x)}$$
$$\exp\left(\frac{\beta + \lambda}{\beta}\right) = \sum_y p^{PT}(y) \exp\left(\frac{1}{\beta} RM(x, y)\right)$$

$Z(x)$ is a normalization constant that requires integrating all possible responses that the model is capable of generating and it is intractable to compute.

It is precisely why reinforcement learning algorithms, such as PPO, is used to compute this theoretical optimal policy.

Define RM by policy

$$p^*(\hat{y}|x) = \frac{1}{Z(x)} p^{PT}(\hat{y}|x) \exp\left(\frac{1}{\beta} RM(x, \hat{y})\right)$$

Since the optimal policy p^* is determined by the reward model RM , we can express the reward model in terms of the policy model:

$$RM(x, \hat{y}) = \beta(\log p^*(\hat{y}|x) - \log p^{PT}(\hat{y}|x)) + \beta \log Z(x) = \beta \log \frac{p^*(\hat{y}|x)}{p^{PT}(\hat{y}|x)} + \beta \log Z(x)$$

Assume that for a language model policy p_θ^{RL} , there is a reward function RM of language evaluation, so that the policy p_θ^{RL} generates samples to maximize the RM . Let a *parameterized* RM be:

$$RM_\theta(x, \hat{y}) \triangleq \beta \log \frac{p_\theta^{RL}(\hat{y}|x)}{p^{PT}(\hat{y}|x)} + \beta \log Z(x) \longleftarrow \text{Problem: Still intractable}$$

↑
Definition, not derivation

Bradley-Terry Model can help

We introduce the Bradley-Terry Model in RLHF lecture. For the same cue word x , the model generates two responses y_w and y_l . Assume the latent reward function is $r(x, y)$. According to the Bradley-Terry model, the probability that a human prefers y_w to y_l is:

$$P(y_w \succ y_l \mid x) = \frac{\exp(r(x, y_w))}{\exp(r(x, y_w)) + \exp(r(x, y_l))} = \frac{1}{1 + \exp(-(r(x, y_w) - r(x, y_l)))}$$

With the Sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ $P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$

Training of the reward model is to minimize its negative log-likelihood:

$$Loss = -\log P(y_w \succ y_l \mid x) = -\log \sigma(r(x, y_w) - r(x, y_l))$$

Put RM definition into Bradley-Terry Model Loss

$$Loss = -\log P(y_w \succ y_l \mid x) = -\log \sigma(r(x, y_w) - r(x, y_l))$$

But now we have a closed-form of the reward model:

$$RM_{\theta}(x, \hat{y}) \triangleq \beta \log \frac{p_{\theta}^{RL}(\hat{y} \mid x)}{p^{PT}(\hat{y} \mid x)} + \beta \log Z(x)$$

The term $\beta \log Z(x)$ is perfectly canceled out:

$$\begin{aligned} RM_{\theta}(x, y_w) - RM_{\theta}(x, y_l) &= \left(\beta \log \frac{p_{\theta}^{RL}(y_w \mid x)}{p^{PT}(y_w \mid x)} + \beta \log Z(x) \right) \\ &\quad - \left(\beta \log \frac{p_{\theta}^{RL}(y_l \mid x)}{p^{PT}(y_l \mid x)} + \beta \log Z(x) \right) \\ &= \beta \log \frac{p_{\theta}^{RL}(y_w \mid x)}{p^{PT}(y_w \mid x)} - \beta \log \frac{p_{\theta}^{RL}(y_l \mid x)}{p^{PT}(y_l \mid x)} \end{aligned}$$

We can directly optimizing the model:

$$L_{DPO}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{p_{\theta}^{RL}(y_w \mid x)}{p^{PT}(y_w \mid x)} - \beta \log \frac{p_{\theta}^{RL}(y_l \mid x)}{p^{PT}(y_l \mid x)} \right) \right]$$

DPO Effect

Zephyr-7B was a landmark model launched by HuggingFaceH4 at the end of 2023. Using only 7B parameters, it surpassed Llama-2-Chat, which had 70B parameters at the time, in dialogue capabilities (such as MT-Bench scores) through highly skillful DPO training.

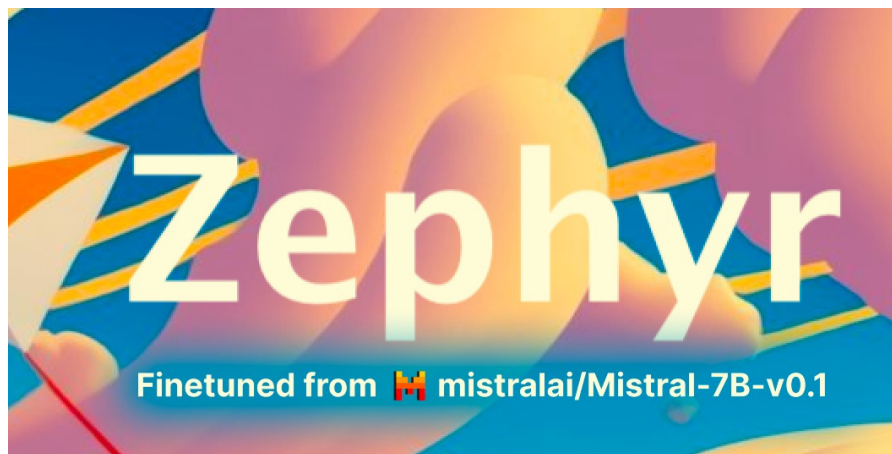


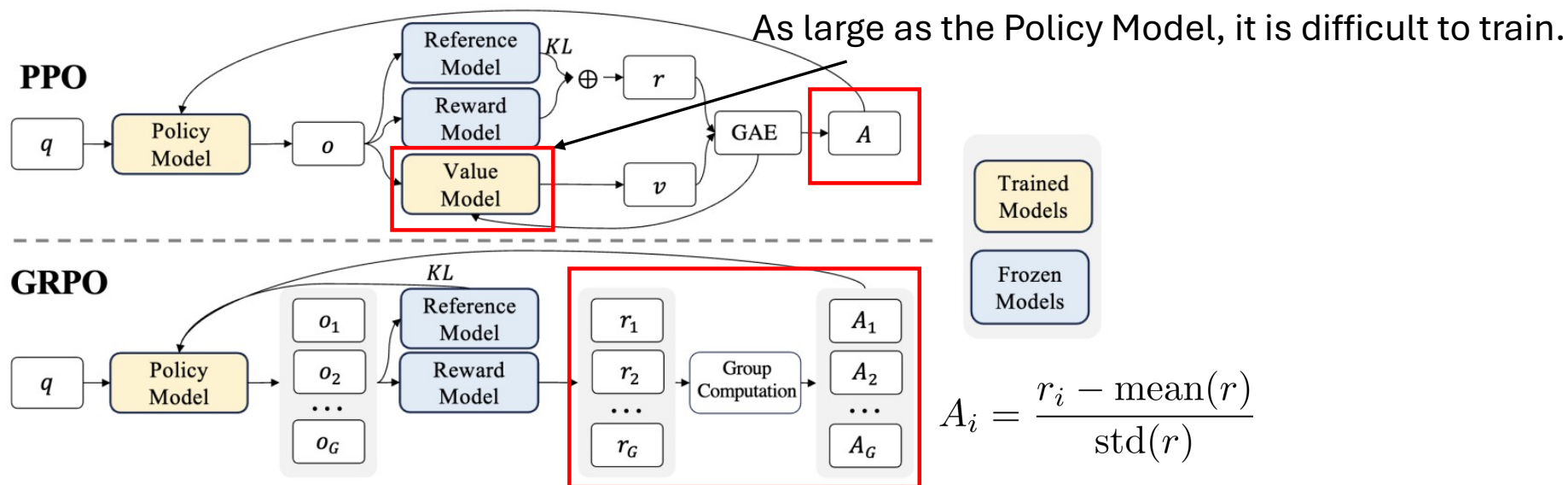
Image source: <https://huggingface.co/HuggingFaceH4/zephyr-7b-beta>

Model	Size	Alignment	MT-Bench (score)	AlpacaEval (win rate %)
StableLM-Tuned- α	7B	dSFT	2.75	-
MPT-Chat	7B	dSFT	5.42	-
Xwin-LMv0.1	7B	dPPO	6.19	87.83
Mistral-Instructv0.1	7B	-	6.84	-
Zephyr-7b- α	7B	dDPO	6.88	-
Zephyr-7b-β	7B	dDPO	7.34	90.60
Falcon-Instruct	40B	dSFT	5.17	45.71
Guanaco	65B	SFT	6.41	71.80
Llama2-Chat	70B	RLHF	6.86	92.66
Vicuna v1.3	33B	dSFT	7.12	88.99
WizardLM v1.0	70B	dSFT	7.71	-
Xwin-LM v0.1	70B	dPPO	-	95.57
GPT-3.5-turbo	-	RLHF	7.94	89.37
Claude 2	-	RLHF	8.06	91.36
GPT-4	-	RLHF	8.99	95.28

GRPO Motivation

PPO: uses a critic (value model) for calculating the advantage. Taking GPU resources.

GRPO: Abandon Critic but just let the peers provide a reference



For the same input question, the current Policy model generates G different answers in parallel. We obtain G specific scores $r_1, r_2, \dots, r_G$, and **the average of these G scores is the natural baseline.**

[4] Shao, et al., "Deepseekmath: Pushing the limits of mathematical reasoning in open language models." arXiv:2402.03300 (2024).

GRPO Loss

The loss formula for GRPO is actually very similar to that of PPO, except that the source of the advantage A_i has changed.

$$J_{GRPO}(\theta) = \mathbb{E}_{q, \{o_i\}} \left[\frac{1}{G} \sum_{i=1}^G \left(\frac{1}{T_i} \sum_{t=1}^{T_i} \min(r_{i,t}(\theta) A_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) A_i) - \beta \mathbb{D}_{KL} \right) \right]$$

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{old}}(o_{i,t} | q, o_{i,<t})}$$

$$L_{clip} = \min \left[r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right]$$

- G : Group size.
- $o_{i,t}$: The t -th token in the i -th sequence.
- $o_{i,<t}$: The prefix context.
- $r_{i,t}(\theta)$: Token-level importance sampling ratio.
- A_i : Advantage score of the i -th response.

[4] Shao, et al., "Deepseekmath: Pushing the limits of mathematical reasoning in open language models." arXiv:2402.03300 (2024).

Industry: PPO vs DPO

Feature / Dimension	PPO (Proximal Policy Optimization)	DPO (Direct Preference Optimization)
Industry Positioning	The Heavyweight's Computing Moat	The Challenger's Open-Source Hero
Key Representatives	OpenAI (GPT-4 era), Anthropic (Claude)	Hugging Face (Zephyr), Mistral, Llama 3
Core Advantage	Global exploration of the "solution space"	Elegant, lightweight, and plug-and-play
Performance Ceiling	Extremely high (Best for complex logic & long text)	Highly effective for general alignment
Resource Requirement	Massive compute & engineering bandwidth	Accessible to open-source developers



Image source: <https://ai.gopubby.com/llm-alignment-with-sft-rlhf-dpo-and-grpo-026333dfaf1f>

DPO Extension 1: Kahneman-Tversky Optimization (KTO)

DPO Problem: DPO relies heavily on strictly paired preference data. However, in real-world scenarios, collecting perfectly aligned paired data is extremely costly, and we often only get a large amount of unpaired data, such as "thumbs up" or "thumbs down".

Solution: Humans are not as sensitive to "receiving good answers" as they are to "avoiding bad answers." Therefore, there is **no need for pairwise comparisons**; we can directly maximize the implicit utility value of each **individual** answer based on whether it is a "praise" or a "dislike."

$$\mathcal{L}_{\text{KTO}}(\pi_{\theta}) = \mathbb{E}_{(x,y)} [w(y) (1 - v(x, y; \beta))] \quad \left\{ \begin{array}{ll} v = \sigma\left(\beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} - z_{\text{ref}}\right) & \text{when } y \text{ is a good answer} \\ v = \sigma\left(-(\beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} - z_{\text{ref}})\right) & \text{when } y \text{ is a bad answer} \end{array} \right.$$

[5] Ethayarajh, Kavin, et al. "Kto: Model alignment as prospect theoretic optimization." arXiv preprint arXiv:2402.01306 (2024).

DPO Extension 2: Simple Preference Optimization (SimPO)

DPO Problem: DPO discovered that by lengthening the generated sentences, even if the advantage of each token is only slight, the cumulative effect can significantly widen the implicit reward gap between good answers and bad answers.

Solution:

$$r_{\text{SimPO}}(x, y) = \frac{\beta}{|y|} \log \pi_{\theta}(y|x)$$

$$\mathcal{L}_{\text{SimPO}} = -\log \sigma(r_{\text{SimPO}}(x, y_w) - r_{\text{SimPO}}(x, y_l) - \gamma)$$

x : The input prompt

y_w, y_l : The chosen and rejected responses

$|y|$: Length of the response (number of tokens)

$\pi_{\theta}(y|x)$: Probability of generating y under current policy

β : Scaling hyperparameter

r_{SimPO} : Length-normalized implicit reward

γ : Reward margin (ensures a strict quality gap)

σ : The logistic sigmoid function

[6] Meng, Yu, Mengzhou Xia, and Danqi Chen. "Simpo: Simple preference optimization with a reference-free reward." Advances in Neural Information Processing Systems 37 (2024): 124198-124235.

GRPO Extension 1: DAPO

GRPO Problem: Standard GRPO uses a **symmetric clipping** to limit update magnitude. This inhibit the model from exploring high-reward young outputs, causing the model to repeat a single pattern.

Scenario (Token Type)	Initial Prob. (π_{old})	Clipped Upper Bound ($\pi_{\text{old}} \times 1.2$)	Actual Max Prob. Allowed	Fatal Impact
A. Exploitation Token	0.90	$0.90 \times 1.2 = 1.08$	nearly 1.0	Unrestricted: Probability easily pushed to 0.999; the model becomes overconfident.
B. Exploration Token	0.01	$0.01 \times 1.2 = 0.012$	0.012	Strictly Bounded: Novel tokens cannot emerge, killing diversity (Entropy Collapse).

Solution: give **higher upper bounds** to encourage low-probability tokens with positive advantages.

$$\mathcal{J}_{\text{DAPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip} \left(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}} \right) \hat{A}_{i,t} \right) \right]$$

[7] Yu, Qiyang, et al. "Dapo: An open-source llm reinforcement learning system at scale." arXiv preprint arXiv:2503.14476 (2025).

Quiz

1. Pen and paper; no compute or cellphone allowed.
 2. Turn in your answer sheet when you leave the classroom.
- Q1 (T/F): advantages is not used when training LLM with RLHF.
 - Q2 (T/F): Due to the comparison between the winner and loser in the optimization of reward model, we don't need to calculate the partition function $Z(x)$.
 - Q3 (T/F): In the GRPO algorithm, multiple samples from different queries are used to calculate the advantage.
 - A: F; T; F